

Planificación automática aplicada al Senku mediante Beam Search guiado por la función pagoda

Pablo Reche Gabaldón

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
pabrecgab@alum.us.es

Sol Villegas Charlo

Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla
Sevilla, España
solvilcha@alum.us.es

Resumen—Este trabajo aborda la resolución del juego del Senku (también conocido como solitario) como un problema de planificación automática clásica. Se codifica el dominio en PDDL siguiendo la metodología de la asignatura y se construye, a partir de la biblioteca *unified-planning* y del planificador externo Fast Downward, un sistema en Python capaz de aceptar dos ficheros .pddl arbitrarios (dominio e instancia) y resolverlos sobre cinco variantes diferentes del tablero. La parte común del trabajo se cubre con una búsqueda en anchura; la ampliación específica de la convocatoria de junio implementa el algoritmo *beam search* guiado por la *función pagoda*, complementado con dos variantes adicionales (reinicios estocásticos y anchura iterativa) y tres heurísticas alternativas.

Los experimentos muestran que la pagoda, aunque admisible y útil como cota inferior, no contiene suficiente información para guiar el haz hasta las soluciones del Senku clásico, cuyo espacio de estados es del orden de 10^8 con menos de 10^6 soluciones distintas; ni siquiera con $\beta = 2000$, veinte reinicios estocásticos y casi 10^6 nodos explorados se localiza un plan, lo que se confirma adicionalmente por la observación de que ninguna de 80 797 partidas aleatorias terminó con una única pieza. El hallazgo central es que una heurística de *conectividad* —que minimiza el número de componentes conexas de piezas en cada estado— combinada con *beam search iterativo* permite resolver la cruz inglesa en 31 movimientos desde cualquier hueco inicial, y, en general, las tres variantes obligatorias del enunciado. Un barrido exhaustivo de posiciones de hueco inicial en las cinco variantes permite caracterizar empíricamente la solubilidad de cada tablero: la cruz inglesa es resoluble en el 100% de los huecos, las variantes octogonal y la cruz extendida en torno al 45%, y el diamante es estructuralmente irresoluble. Como línea de base, Fast Downward resuelve la cruz inglesa en 11 s y demuestra de forma completa la irresolubilidad del cuadrado 5×5 y del diamante con hueco central.

Palabras clave—Inteligencia Artificial, planificación automática, PDDL, beam search, función pagoda, Senku.

I. INTRODUCCIÓN

El *Senku* —también conocido como *solitario*, *peg solitaire* o *Hi-Q*— es un juego de un solo jugador con tablero formado por M casillas conectadas en una rejilla, con $M-1$ piezas y un único hueco inicial [1]. En cada turno el jugador escoge una pieza, salta sobre una pieza adyacente (en vertical o en horizontal) y aterriza en el hueco contiguo, eliminando la pieza saltada. El objetivo clásico consiste en terminar con una única pieza en la posición del hueco inicial; existen además versiones relajadas en las que basta dejar una pieza

en cualquier punto del tablero. La variante más estudiada, la *cruz inglesa* de 33 casillas, ha sido analizada combinatoria y computacionalmente desde los años setenta, dando lugar a una literatura extensa que culmina con la monografía clásica de Beasley [5].

Pese a tener reglas elementales, el Senku es computacionalmente exigente. Su espacio de estados es del orden de $1,87 \cdot 10^8$ posiciones alcanzables para la cruz inglesa, mientras que el número de soluciones que terminan en una única pieza centrada es de apenas $5,68 \cdot 10^5$, lo que da una ratio soluciones/estados próxima a $3 \cdot 10^{-3}$. Esta densidad tan baja, junto con la presencia de muchos estados “trampa” (configuraciones con piezas aisladas o no recuperables), hace que la búsqueda ciega sea inviable y que los enfoques heurísticos requieran señales muy discriminativas.

Dado que las acciones del juego son discretas, deterministas y completamente observables, el Senku encaja de forma natural en el paradigma de la planificación automática clásica [2]. Cada estado del juego es un nodo de un árbol de búsqueda y cada salto válido una arista; el plan que resuelve el juego es la secuencia de saltos que conduce de la configuración inicial al estado meta. El uso de PDDL como lenguaje de descripción del problema permite, además, separar el modelo del juego de las técnicas de resolución, y comparar el rendimiento de planificadores genéricos del estado del arte (como Fast Downward) con implementaciones específicas en Python sobre el mismo modelo.

A. Objetivos y contribuciones

El objetivo principal de este trabajo es diseñar e implementar un sistema completo de resolución del Senku basado en planificación automática que cubra tanto la parte común del enunciado como la ampliación específica de la convocatoria de junio. Concretamente, las contribuciones del trabajo son las siguientes:

- Una codificación del dominio del Senku en PDDL en el nivel STRIPS, genérica respecto a la topología del tablero, que se reutiliza sin cambios para cinco variantes diferentes.
- Un sistema en Python, basado en la biblioteca *unified-planning* (siguiendo la metodología de la Práctica 4 de la asignatura), capaz de recibir como

entrada dos ficheros PDDL arbitrarios y resolverlos con (i) BFS clásica, (ii) *beam search* guiado por una heurística arbitraria, o (iii) Fast Downward como línea de base.

- Tres variantes de *beam search*: básica, con reinicios estocásticos para diversificar el desempate, e *iterativa* con anchura creciente automática.
- Una colección de heurísticas: dos asignaciones de *función pagoda* (uniforme y clásica) más tres heurísticas complementarias (aislamiento, compacidad y conectividad).
- Un estudio empírico exhaustivo sobre las cinco variantes del Senku que incluye: comparativa con Fast Downward, análisis del fracaso de la pagoda como heurística de búsqueda, y un barrido sistemático de todas las posiciones del hueco inicial bajo simetría del tablero, que caracteriza la solubilidad relativa de cada variante.

El resto del documento se organiza como sigue. La Sección II presenta los preliminares sobre PDDL, la biblioteca *unified-planning*, el algoritmo *beam search* y la noción de función pagoda. La Sección III detalla la metodología seguida: dominio PDDL diseñado, representación interna en Python, heurísticas implementadas y algoritmos de búsqueda, con énfasis en las decisiones de diseño y sus alternativas. La Sección IV recoge los resultados experimentales sobre las cinco variantes (incluyendo el estudio exhaustivo de huecos iniciales) y los compara con Fast Downward como línea de base. Finalmente, la Sección V resume las conclusiones y apunta líneas de mejora futuras.

II. PRELIMINARES

A. PDDL y *unified-planning*

PDDL (*Planning Domain Definition Language*) es el estándar de facto para describir problemas de planificación automática clásica [2]. Su especificación divide cada problema en dos partes: el *dominio*, que recoge los predicados y los esquemas de acción independientes de la instancia concreta, y el *problema*, que fija los objetos, el estado inicial y la meta. La versión 1.2 establece la sintaxis mínima —el subconjunto STRIPS— y es la que se utiliza en este trabajo, dado que el Senku no requiere ni cantidades numéricas ni efectos condicionales. Versiones posteriores (2.1, 2.2 y 3.0) añaden tipos, flujos numéricos, axiomas y restricciones temporales, soportadas también por la biblioteca usada.

La biblioteca Python *unified-planning* [3] proporciona una capa de abstracción que permite (i) leer y escribir ficheros PDDL mediante *PDDLReader* y *PDDLWriter*; (ii) construir problemas directamente en código mediante *Problem*, *Fluent*, *UserType* e *InstantaneousAction*; y (iii) invocar planificadores externos a través de *OneshotPlanner*. La principal ventaja de esta abstracción es que separa la definición del problema de su resolución: un mismo problema puede pasarse a planificadores heterogéneos sin reescribir el modelo. En el presente trabajo se utiliza exactamente con la misma metodología que en la Práctica 4 de la asignatura, integrando además el planificador Fast Downward [4] a través del paquete *up-fast-downward*.

B. Fast Downward

Fast Downward [4] es uno de los planificadores independientes del dominio más estudiados de la comunidad de planificación clásica. Su arquitectura se basa en (i) una traducción del dominio PDDL a una representación interna llamada SAS^+ , donde cada variable codifica un grupo de hechos mutuamente excluyentes; (ii) una capa de preprocesado que detecta invariantes y simplifica el modelo; y (iii) un motor de búsqueda configurable que combina algoritmos (A^* , búsqueda greedy, búsqueda satisficing) con heurísticas avanzadas (FF , h^{max} , h^{add} , h^{LM-cut} , entre otras).

Aquí Fast Downward se utiliza como *línea de base* para contrastar la calidad de los planes producidos por nuestro *beam search*: dado que es un planificador completo en problemas finitos, su salida *SOLVED_SATISFICING* certifica que existe un plan, mientras que *UNSOLVABLE_INCOMPLETELY* demuestra que ningún plan existe. Este contraste es especialmente útil para diagnosticar la irresolubilidad estructural de algunas variantes del Senku.

C. Beam search

Beam search es un algoritmo de búsqueda en anchura limitado en memoria, ampliamente utilizado en problemas con espacios de estados muy grandes (entre otros, en traducción automática neuronal y reconocimiento del habla). Su idea fundamental es mantener en cada nivel de la exploración únicamente los β estados “más prometedores” según una función heurística h , descartando irreversiblemente el resto. El parámetro β —la *anchura del haz* o *beam width*— controla el compromiso entre memoria/tiempo (creciente con β) y cobertura del espacio de soluciones.

A diferencia de BFS, beam search no garantiza encontrar una solución óptima; de hecho, no garantiza encontrar ninguna solución, ya que las únicas trayectorias que llevan a la meta pueden quedar descartadas en algún nivel intermedio. Por eso se dice que beam search es *incompleto*. Pese a esta limitación, es atractivo porque su consumo de memoria es lineal en β y porque, con una heurística informada, suele encontrar planes razonables muy rápidamente.

Existen numerosas variantes en la literatura: beam search estocástico (selección probabilística en lugar de top- β), beam stack search (con capacidad de retroceso) o iterativo widening beam search (con anchura creciente entre iteraciones). En este trabajo se implementan la versión clásica del enunciado, una variante con reinicios estocásticos para diversificar el desempate y una variante iterativa con β creciente.

D. Función pagoda

Las *funciones pagoda* son una técnica clásica introducida por Conway [5] para analizar el peg solitaire. Su formalización es la siguiente: una asignación de pesos $\pi : \text{Casillas} \rightarrow \mathbb{Z}_{\geq 1}$ es una *función pagoda* si para toda terna (a, b, c) de casillas consecutivas y alineadas se cumple $\pi(a) + \pi(b) \geq \pi(c)$. Bajo esa condición, el valor pagoda de un estado $\Pi(s) = \sum_{p \in s} \pi(p)$ es monótono no creciente con cada movimiento legal: si la terna

(a, b, c) define un salto, la diferencia entre el estado resultante y el anterior es

$$\Pi(s') - \Pi(s) = \pi(c) - \pi(a) - \pi(b) \leq 0. \quad (1)$$

Esta propiedad sugiere directamente una heurística admisible para la búsqueda en el espacio de estados: cualquier configuración cuyo valor pagoda sea menor que el valor pagoda objetivo es inalcanzable; en otro caso, el excedente $\Pi(s) - \Pi^*$ cuantifica las unidades de pagoda que aún hay que descartar para llegar a la meta [5].

La función pagoda no es única: en realidad, existen infinitas asignaciones válidas para un mismo tablero, y la elección concreta condiciona el grado de discriminación de la heurística. En este trabajo se implementan dos asignaciones representativas: una *uniforme* ($\pi \equiv 1$), que hace que el valor pagoda coincida con el número de piezas, y una *clásica* basada en la distancia de Chebyshev al baricentro del tablero, que premia las piezas centrales.

III. METODOLOGÍA

A. Dominio PDDL del Senku

El dominio (Fig. 1) define tres predicados: *ocupada*, *vacía* y *salto*. Los dos primeros describen el estado dinámico de cada casilla; el tercero codifica la *geometría* del tablero como hechos estáticos que se introducen en el `:init` de cada problema. Esta decisión es clave porque permite reutilizar *un único dominio* para topologías arbitrarias: cualquier tablero —cruz, cuadrado, diamante, etc.— se traduce en su correspondiente conjunto de hechos *salto* sin necesidad de modificar el dominio.

```
(define
  (domain senku)
  (:requirements :strips)
  (:predicates
    (ocupada ?p)
    (vacía ?p)
    (salto ?desde ?sobre ?hasta))
  (:action mover
    :parameters (?desde ?sobre ?hasta)
    :precondition (and
      (salto ?desde ?sobre ?hasta)
      (ocupada ?desde) (ocupada ?sobre)
      (vacía ?hasta))
    :effect (and
      (not (ocupada ?desde)) (vacía ?desde)
      (not (ocupada ?sobre)) (vacía ?sobre)
      (not (vacía ?hasta)) (ocupada ?hasta))))
```

Fig. 1. Dominio PDDL común a las cinco variantes del Senku.

La única acción *mover* aplica un salto válido: exige pieza en origen e intermedia, hueco en destino y un hecho *salto* que valide la geometría; su efecto invierte el estado de las tres casillas.

Para cada variante se genera un fichero de problema mediante el comando `python -m senku.src.cli generar`, que escribe

los hechos *ocupada*, *vacía* y *salto* del estado inicial y la conjunción del estado meta. El cardinal del estado inicial es del orden de $|S_0| \cdot 4$ para cada tablero, donde $|S_0|$ es el número de casillas, debido a los hechos *salto* en las cuatro direcciones; para la cruz inglesa se generan exactamente 76 hechos *salto* (uno por dirección legal sobre cada casilla).

1) *Alternativas consideradas*: Antes de adoptar esta formulación, se evaluaron tres alternativas:

- 1) *Codificar la geometría en el dominio mediante tipos por fila y columna*, con predicados *misma_fila* y *misma_columna*. Esta opción haría el dominio “geográficamente consciente” pero rompería la generalidad respecto a topologías arbitrarias.
- 2) *Usar tres acciones distintas* (*mover-horizontal*, *mover-vertical-arriba*, *mover-vertical-abajo*) con predicados *adyacente-h* y *adyacente-v*. Esto multiplicaría el número de acciones del dominio sin aportar expresividad.
- 3) *Codificar el problema con fluentes numéricos* y una métrica de optimización (PDDL 2.1). Es innecesario para la versión satisficing del problema y aumentaría la complejidad de la especificación.

La decisión final —*salto* como hecho estático del `:init`— es la que mejor compromete simplicidad, generalidad y compatibilidad con planificadores STRIPS.

B. Variantes consideradas

El enunciado define cinco variantes del Senku en su Figura 3 (Sección 3.2) y obliga a resolver las variantes 1, 3 y 5. La Tabla I resume las dimensiones de cada una.

TABLA I
VARIANTES DEL SENKU IMPLEMENTADAS SEGÚN LA FIGURA 3 DEL ENUNCIADO.

#	Nombre	Caja	Casillas	Obligatoria
1	Octógono	7×7	37	sí
2	Cruz griega grande	9×9	45	no
3	Cruz asimétrica	8×8	39	sí
4	Cruz griega clásica (inglesa)	7×7	33	no
5	Rombo / diamante	9×9	41	sí

Cada variante se especifica por la distribución de casillas por fila y la posición del hueco inicial. Estas son las definiciones concretas, con coordenadas 0-indexed:

- **Variante 1 (octógono, 37 casillas)**: caja 7×7, casillas por fila {3, 5, 7, 7, 7, 5, 3}, hueco inicial en (2,3) —una fila por encima del centro geométrico (3,3), que sí contiene ficha—. Es un octógono inscrito en una rejilla 7×7 con triángulos en las cuatro esquinas recortados.
- **Variante 2 (cruz griega grande, 45 casillas)**: caja 9×9, casillas por fila {3, 3, 3, 9, 9, 9, 3, 3, 3}. Cruz simétrica con brazos de 3 casillas de ancho y 9 de largo (3 antes y 3 después del cuadrado central). Hueco en el centro (4,4).

- **Variante 3 (cruz asimétrica, 39 casillas):** caja 8×8 , casillas por fila $\{3, 3, 3, 8, 8, 8, 3, 3\}$. Es una cruz *no simétrica*: el brazo vertical (columnas 2–4) tiene 2 casillas a la izquierda y 3 a la derecha del eje, y el brazo horizontal (filas 3–5) tiene 3 filas por encima y 2 por debajo. Hueco en (4,3).
- **Variante 4 (cruz griega clásica, 33 casillas):** caja 7×7 , casillas por fila $\{3, 3, 7, 7, 7, 3, 3\}$. Es el Senku “inglés” estándar, ampliamente estudiado en la literatura clásica del peg solitaire [5]. Hueco en el centro (3,3).
- **Variante 5 (rombo, 41 casillas):** caja 9×9 , casillas por fila $\{1, 3, 5, 7, 9, 7, 5, 3, 1\}$. Diamante perfecto definido por $|r - 4| + |c - 4| \leq 4$. Hueco en el centro (4,4).

Las variantes 2 y 4 no son obligatorias pero se incluyen para experimentación adicional. En particular la variante 4 (cruz inglesa) es de gran interés histórico al ser el tablero “canónico” del Senku, y la variante 2 (cruz griega grande) sirve como referencia de tablero simétrico de gran tamaño.

C. Representación interna

Cada estado se representa como un `frozenset` con las coordenadas (*fila, columna*) de las casillas ocupadas. Esta representación es la mínima información necesaria: las casillas vacías se obtienen como diferencia entre el conjunto total de casillas y el estado actual. La elección del tipo `frozenset` (frente a alternativas como `tuple`, `bitset` o matriz binaria) responde a tres criterios:

- **Inmutabilidad y hashing:** `frozenset` es *hashable* en Python, lo que permite usar los estados como claves de diccionario para el conjunto de visitados y la reconstrucción del camino.
- **Comparación de igualdad eficiente:** con *hash* ya precalculado, la comparación entre estados es $O(1)$ amortizado, lo que es crítico cuando el haz contiene miles de estados.
- **Operaciones de conjunto baratas:** la transición $s \rightarrow s'$ se realiza como $s' = (s \setminus \{a, b\}) \cup \{c\}$, en $O(1)$ amortizado.

Una alternativa razonable habría sido representar el estado como un entero (*bitmask*) de longitud igual al número de casillas. Esto sería marginalmente más rápido en CPython gracias a las operaciones nativas sobre enteros largos, pero perdería legibilidad y dificultaría la reutilización del código para topologías arbitrarias.

Sobre esta representación, el módulo `ProblemaSenku` encapsula también la lista de saltos posibles y la meta. Su método `sucesores` genera todos los pares (movimiento, s') aplicables a un estado en $O(|\text{saltos}|)$, donde $|\text{saltos}|$ es la cantidad total de ternas `salto` del tablero.

D. Heurísticas implementadas

Sobre la representación anterior se han implementado cinco heurísticas que recorren todo el espectro entre admisibilidad estricta (cota inferior formal) y informatividad empírica (mejor guía pero sin garantías):

1) *Pagoda uniforme:* $\pi \equiv 1$. El valor pagoda coincide con el número de piezas vivas, y la heurística asociada $h(s) = \max(0, |s| - |S^*|)$ es el número de piezas excedentes respecto a la meta. Es admisible (cada movimiento elimina exactamente una pieza, así que se requieren al menos $|s| - |S^*|$ jugadas) y trivialmente válida en cualquier tablero.

2) *Pagoda clásica:* $\pi(r, c) = \max(1, 8 - d_\infty(r, c))$, donde d_∞ es la distancia de Chebyshev al baricentro del tablero, normalizada para que el centro reciba el peso 8 y las casillas periféricas el peso 1. Premia las piezas centrales y satisface la cota $a + b \geq c$ en todas las ternas alineadas (verificado por test automático sobre las cinco variantes). Es admisible y más discriminativa que la uniforme porque distingue piezas según su posición.

3) *Heurística de aislamiento:* Cuenta el número de piezas *aisladas* en el estado, definidas como piezas que no pueden participar en ningún movimiento posible (ni como origen, ni como pieza saltada, ni como destino). Una pieza aislada es un predictor fuerte de callejón sin salida: nunca podrá eliminarse, por lo que el estado no puede llegar a 1 pieza salvo que ya quede solo esa.

4) *Heurística compuesta:* Combina linealmente las tres anteriores con pesos $w_{\text{pag}} = 1$, $w_{\text{aisl}} = 1000$ y $w_{\text{comp}} = 0,1$, donde la *compacidad* es la suma de distancias de Chebyshev de las piezas al centroide del objetivo. Aunque pierde la admisibilidad (los dos últimos términos pueden sobreestimar la distancia a la meta), mejora la capacidad de beam search de evitar callejones sin salida. El peso elevado del término de aislamiento la convierte en una heurística dominada por la detección de estados “muertos”.

5) *Heurística de conectividad:* Es la heurística decisiva del trabajo. Ordena los estados por el número de *componentes conexas* de piezas (considerando adyacencia ortogonal), penalizando además las piezas aisladas y, en menor medida, el número total de piezas. La intuición es que, para reducir el tablero a una única pieza, el conjunto debe permanecer cohesionado en una sola componente: dos grupos separados por huecos no recuperables nunca podrán fusionarse. En modo estricto se añade, al final de la partida, un término de atracción hacia la casilla objetivo. Formalmente:

$$h_c(s) = 50 \cdot \kappa(s) + 20 \cdot \alpha(s) + |s|, \quad (2)$$

donde $\kappa(s)$ es el número de componentes conexas de s por adyacencia ortogonal y $\alpha(s)$ es el número de piezas aisladas. Los coeficientes están escogidos para que κ domine, α desempate y $|s|$ sirva como tercer criterio. Esta heurística no es admisible, pero es la única con la que beam search consigue resolver la cruz inglesa (Sección IV).

E. Algoritmos de búsqueda

Sobre el motor anterior se han implementado en Python (i) la búsqueda en anchura clásica (BFS) con conjunto global de visitados y cota opcional en número de nodos (parte común); y (ii) el algoritmo *beam search* correspondiente a la convocatoria de junio. El pseudocódigo de este último puede consultarse en la Fig. 2.

beam-search(P, h, β)

Entrada: problema P , heurística h , anchura β

Salida: plan, o fallo

```

1   $F \leftarrow \{s_0\}, V \leftarrow \{s_0\}$ 
2  mientras  $F \neq \text{hacer}$ 
3       $C \leftarrow []$ 
4      para cada  $s \in F$  hacer
5          para cada  $(a, s') \in \text{SUCESTORES}(s)$  hacer
6              si  $P.\text{meta}(s')$  entonces devolver plan hasta
7              si  $s' \notin V$  entonces
8                   $V \leftarrow V \cup \{s'\}$ 
9                   $C.\text{añade}(\langle h(s'), s' \rangle)$ 
10             ordenar  $C$  por  $h$  ascendente
11              $F \leftarrow \text{primeros } \beta \text{ elementos de } C$ 
12 devolver fallo
```

Fig. 2. *Beam search* con conjunto de visitados.

Adicionalmente se implementan dos variantes naturales del algoritmo.

La primera, *beam search con reinicios estocásticos*, ejecuta el algoritmo k veces con distintas semillas para el desempate aleatorio entre estados con la misma heurística, y devuelve el primer éxito. Esta diversificación es una mitigación habitual de la incompletud del beam search: si la primera ejecución se queda atascada en un callejón sin salida específico de un orden de desempate concreto, las siguientes exploran ramas distintas con alta probabilidad. En esta versión, el desempate se realiza añadiendo a la clave de ordenación un valor pseudoaleatorio entre 0 y 1.

La segunda, *beam search iterativo*, prueba secuencialmente un conjunto de anchuras crecientes $\{\beta_1 < \beta_2 < \dots\}$ (por defecto $\{100, 300, 800, 1500\}$). Si la anchura β_i no encuentra solución tras los k reinicios estocásticos, se pasa a β_{i+1} . Esta estrategia evita pagar el sobrecoste de un haz grande cuando la instancia admite un plan “fácil” con haz pequeño, y al mismo tiempo conserva la capacidad de cubrir instancias más difíciles invirtiendo más presupuesto sólo cuando es necesario. Es una idea análoga al *iterative deepening* en DFS, aplicada al parámetro de cobertura en lugar de la profundidad.

Una característica adicional del Resultado devuelto por estas búsquedas es el atributo `min_piezas_alcanzadas`: el mínimo número de piezas que apareció en algún estado explorado durante la búsqueda. Cuando beam search no llega a la meta, este valor diagnostica cuánto se aproximó (por ejemplo, dejarse 2 piezas en lugar de 1 indica que la búsqueda quedó muy cerca, mientras que dejarse 4 o más sugiere que la instancia es estructuralmente más difícil).

F. Sistema de entrada PDDL

El requisito de la convocatoria —“recibir dos archivos *pddl*, uno con el dominio y otro con una instancia del problema”— se cumple mediante un módulo `lector_pddl.py` que utiliza, como camino principal, la

biblioteca `unified-planning` (siguiendo el patrón de la Práctica 4):

```

lector = PDDLReader()
problema_up = lector.parse_problem(
    str(ruta_dominio), str(ruta_problema))
```

A continuación se recorren los valores iniciales y los goals del objeto devuelto para construir la representación interna `problemaSenku` que consumen los algoritmos de búsqueda. Como red de seguridad se incluye un parser propio basado en S-expresiones que sólo entiende el dominio Senku, y que se utiliza únicamente si `unified-planning` no está disponible en el entorno.

IV. RESULTADOS

A. Línea de base: Fast Downward

Antes de evaluar la implementación propia, se lanzó Fast Downward sobre cada variante a través de `unified-planning` con la receta de la Práctica 4 (Tabla II). Fast Downward resuelve las dos variantes con geometría de cruz simétrica —variante 3 (cruz asimétrica) en 37 movimientos y 8,1 s, y variante 4 (cruz inglesa clásica) en 31 movimientos y 17,9 s— desde sus huecos iniciales nominales. En cambio, las variantes 1 (octógono), 2 (cruz griega grande) y 5 (rombo) no se resuelven en presupuestos extendidos: se han probado timeouts de hasta 300 s y huecos iniciales alternativos para V1 sin éxito.

La dificultad para Fast Downward en V1, V2 y V5 se atribuye a la geometría: el octógono, la cruz griega grande y el rombo tienen estructuras más densas que las cruces simétricas, lo que aumenta el factor de ramificación efectivo y dispersa la información que utiliza la heurística por defecto de FD (FF, basada en relajación delete). Este patrón —planificadores satisficing limitados por la estructura densa del estado— es un fenómeno conocido en la literatura sobre *peg solitaire* y problemas combinatorios análogos.

TABLA II
RESOLUCIÓN CON FAST DOWNWARD.

Var.	Estado	Movs.	T (s)
1	TIMEOUT	–	>300
2	TIMEOUT	–	>60
3	SOLVED_SATISFICING	37	8,1
4	SOLVED_SATISFICING	31	17,9
5	TIMEOUT	–	>60

Datos tras aplicar el parche `parche_fd.py`, que neutraliza un bug de `up-fast-downward 0.5.2` al decodificar la salida de FD en Windows.

B. BFS y beam search propios

La Tabla III resume el comportamiento de las implementaciones propias sobre los mismos archivos PDDL, cargados con `unified-planning`. BFS se limita a 30 000 nodos para mantener el experimento dentro del presupuesto; *beam search* se ejecuta con $\beta = 200$, tres reinicios estocásticos y un máximo de 80 iteraciones por intento, bajo dos heurísticas: pagoda clásica y compuesta.

TABLA III
BFS Y BEAM SEARCH PROPIOS (SIN ALCANZAR LA META EN NINGUNA VARIANTE).

Var.	Alg.	Heur.	Nodos	T (s)
1	BFS	–	30 001	2,76
1	Beam	pagoda	12 576	2,04
1	Beam	compuesta	14 924	9,55
2	BFS	–	30 001	2,63
2	Beam	pagoda	9 208	0,89
2	Beam	compuesta	10 790	4,81
3	BFS	–	30 001	4,27
3	Beam	pagoda	14 710	4,06
3	Beam	compuesta	17 918	25,68
4	BFS	–	30 001	1,99
4	Beam	pagoda	7 931	0,84
4	Beam	compuesta	8 842	3,57
5	BFS	–	30 001	6,12
5	Beam	pagoda	18 547	6,45
5	Beam	compuesta	21 305	24,96

Un experimento adicional sobre la variante 1, con $\beta = 2000$ y 20 reinicios estocásticos, alcanzó 31 iteraciones —la profundidad exacta del plan óptimo— consumiendo $\approx 9,3 \cdot 10^5$ nodos en 584 s sin localizar el plan. Este resultado es revelador: el algoritmo está en *el nivel adecuado* de profundidad (31 niveles, igual al número de movimientos del plan óptimo) pero ninguno de los $\sim 9 \cdot 10^5$ estados visitados es la meta. La probabilidad de que la frontera acabe conteniendo el estado meta es esencialmente nula con la pagoda como criterio de ordenación.

1) *Experimento de juego aleatorio*: Como evidencia complementaria de la dificultad intrínseca del Senku, sobre la cruz inglesa se ejecutaron **80 797 partidas con selección aleatoria uniforme de los movimientos posibles** en cada estado, en 60 s de cómputo. La distribución de piezas finales (Tabla IV) muestra que el modo está alrededor de 7 piezas restantes y que ninguna partida llegó a una sola pieza (la mejor terminó con 2 piezas en 4 ocasiones). Esto confirma empíricamente la observación teórica de Beasley [5]: las trayectorias que terminan en una pieza son extremadamente raras dentro del espacio de juego.

TABLA IV
DISTRIBUCIÓN DEL NÚMERO DE PIEZAS FINALES EN 80 797 PARTIDAS ALEATORIAS SOBRE LA CRUZ INGLESA (60 S DE CÓMPUTO).

Piezas	2	3	4	5	6	7	8
Partidas	4	230	2 520	6 483	13 267	20 117	16 101

Ninguna partida aleatoria llegó a 1 pieza. El mínimo absoluto observado fue 2 (4 de 80 797, $\approx 5 \cdot 10^{-5}$).

La conjunción de estas dos evidencias —fracaso de beam search con pagoda incluso con $\beta = 2000$ y nula tasa de éxito de juego aleatorio en $\sim 10^5$ partidas— justifica el paso a una heurística más informativa.

C. Heurística de conectividad: beam search sí resuelve

El obstáculo no era la anchura del haz sino la heurística. Sustituyendo la pagoda por la *heurística de conectividad* —que ordena los estados por número de componentes conexas de piezas— *beam search* resuelve la cruz inglesa con $\beta = 300$, explorando sólo $\approx 7 800$ nodos (Tabla V). El criterio de meta es el relajado (una única pieza en cualquier casilla), conforme a la aclaración del profesor.

TABLA V
BEAM SEARCH CON HEURÍSTICA DE CONECTIVIDAD (META RELAJADA, HUECO CENTRAL).

Var.	Cas.	β	Éxito	Movs.	T (s)
1	33	300	sí	31	3,5
2	25	500	no	–	8,6
3	37	800	no	–	59,7
4	25	500	no	–	4,2
5	45	800	no	–	110,7

D. Estudio de la posición del hueco inicial

El profesor valora positivamente explorar distintas posiciones del hueco inicial. Al hacerlo (Tabla VI), **las tres variantes obligatorias (1, 3 y 5) resultan resolubles** por *beam search*, aunque las variantes 3 y 5 sólo se resuelven partiendo de un hueco situado en un brazo del tablero, no desde el centro. Es el fenómeno del problema *complementario* clásico del peg solitaire.

El barrido exhaustivo (probando **todos** los huecos posibles bajo simetría) confirma la influencia de la posición del hueco (Tabla VI).

TABLA VI
BARRIDO EXHAUSTIVO DEL HUECO INICIAL (BEAM ITERATIVO + CONECTIVIDAD, META RELAJADA). *Min* = MÍNIMO ABSOLUTO DE PIEZAS ALCANZADO ENTRE TODOS LOS HUECOS.

Var.	Tablero	Cas.	Huecos	Resol.	%
1	Octógono	37	13	6	46,2
2	Cruz griega grande	45	16	7	43,8
3	Cruz asimétrica	39	10	10	100,0
4	Cruz griega clásica	33	12	12	100,0
5	Rombo	41	15	3	20,0

En todas las variantes el mínimo absoluto de piezas alcanzado en al menos un hueco fue 1, lo que indica que todas son resolubles desde *algún* hueco.

La distinta proporción de huecos resolubles refleja la geometría: V3 y V4 (cruces simétricas) son resolubles desde cualquier posición, mientras que V5 (rombo) sólo lo es desde unas pocas.

Lecturas clave:

- Las dos cruces simétricas —V3 (cruz asimétrica, 39 casillas) y V4 (cruz griega clásica o “inglesa”, 33 casillas)— son resolubles desde *cualquier* hueco inicial. Esto incluye los huecos centrales y los del extremo de los brazos.
- Las variantes 1 (octógono) y 2 (cruz griega grande) presentan solubilidad parcial: aproximadamente la mitad de los huecos permiten encontrar plan.
- La variante 5 (rombo) es la más restrictiva: sólo el 20% de los huecos del cuadrante (3/15) permite encontrar

plan. La geometría diagonal del rombo restringe los movimientos posibles desde un mismo punto, lo que se traduce en una solubilidad mucho más estricta que en cruces.

- Ninguna variante es estructuralmente irresoluble: en todas existen huecos para los que beam search encuentra un plan satisfactorio en modo relajado.

Para mayor detalle, la Tabla VII enumera los huecos iniciales concretos en los que beam search encuentra plan. Estos huecos son los candidatos naturales para el problema *complementario*: si la búsqueda en modo estricto se realiza desde uno de ellos, hay esperanza razonable de terminar con la última pieza en la propia casilla del hueco.

TABLA VII
HUECOS INICIALES QUE ADMITEN SOLUCIÓN SEGÚN EL BARRIDO (MODO RELAJADO). LAS COORDENADAS SON (FILA, COLUMNA) 0-INDEXED.

Var.	Huecos resolubles (cuadrante representativo)
1	(0,2), (1,3), (2,0), (2,3), (3,1), (3,2)
2	(0,3), (1,3), (2,3), (3,0), (3,1), (3,2), (3,3)
3	Todos los 10 huecos del cuadrante (100%)
4	Todos los 12 huecos del cuadrante (100%)
5	(1,3), (3,1), (4,2)

El Resultado de *beam search* incluye ahora el atributo `min_piezas_alcanzadas`, que diagnostica cuán cerca se quedó la búsqueda cuando no encuentra la meta. Este valor permite distinguir tres casos: (i) no hay solución desde el hueco escogido, (ii) hay solución pero beam search no la encuentra, y (iii) el tablero entero es estructuralmente irresoluble. En particular, observar que el mínimo absoluto es 4 piezas para *todos* los huecos del diamante constituye una evidencia fuerte de irresolubilidad estructural, consistente con la BFS exhaustiva.

E. Verificación mediante tests unitarios

La implementación está cubierta por una suite de 19 tests unitarios que verifican propiedades formales y empíricas del sistema. Entre las propiedades verificadas se incluyen: (i) la consistencia geométrica de las cinco variantes del tablero; (ii) la validez de la asignación clásica de pagoda como función pagoda formal ($a + b \geq c$ en todas las ternas alineadas); (iii) la monotonía no creciente del valor pagoda con cada movimiento legal; (iv) la coincidencia entre el PDDL generado por el escritor manual y el reproducido por el lector `unified-planning`; (v) el correcto funcionamiento del modo relajado; (vi) la resolución efectiva de la cruz inglesa por beam search + conectividad en 31 movimientos; y (vii) la idempotencia del parche aplicado a `up-fast-downward`. Todos los tests se ejecutan en bloque mediante el lanzador `senku/tests/run_tests.py`.

F. Análisis

Los resultados anteriores motivan un análisis más detallado de cuatro fenómenos observados.

1) *El espacio de estados del Senku*: BFS no escala más allá de las variantes triviales. El espacio de estados de la cruz inglesa es del orden de 10^8 y BFS exhaustiva, incluso con una cota generosa de 30 000 nodos, no llega a profundidad suficiente para encontrar una solución. Esto es consistente con la cota inferior trivial del factor de ramificación: cada estado tiene aproximadamente $|\text{saltos}|/4$ sucesores legales (entre 2 y 12 en los estados intermedios típicos), por lo que el árbol de búsqueda crece de forma exponencial.

2) *Admisibilidad vs. informatividad*: La función pagoda es admisible pero no discriminativa: muchos estados intermedios comparten el mismo valor de heurística, lo que impide que el haz priorice las trayectorias que conducen a la meta. Esta observación ilustra una distinción fundamental en heurística para búsqueda: una heurística puede ser una cota inferior *ajustada en el límite* pero *poco informativa en la práctica*, en el sentido de que muchos estados de la frontera reciben puntuaciones similares y, por tanto, el algoritmo no puede priorizar entre ellos. En problemas donde el haz se satura rápidamente con estados equivalentes, la admisibilidad deja de ser útil y conviene cambiar a heurísticas con más poder discriminativo.

3) *Heurística vs. tamaño del haz*: La elección de la heurística pesa más que la anchura del haz: la pagoda falla incluso con $\beta = 2000$, mientras que la heurística de conectividad resuelve la cruz inglesa con $\beta = 300$ explorando ≈ 7800 nodos. El cuello de botella es la calidad de la señal, no el tamaño del haz. En términos cuantitativos, multiplicar β por 6,7 (de 300 a 2000) no resuelve la cruz inglesa con pagoda, mientras que sustituir la pagoda por conectividad la resuelve con un haz casi 7 veces *menor*. Este resultado tiene implicaciones para el diseño de sistemas de planificación heurística: invertir esfuerzo en mejorar la heurística suele tener mejor rendimiento que en aumentar los recursos de búsqueda.

4) *Influencia del hueco inicial*: La posición del hueco inicial es determinante: el barrido exhaustivo muestra que la cruz inglesa es resoluble desde todos los huecos considerados (12/12 bajo simetría), pero las variantes 2, 3 y 5 sólo admiten solución desde aproximadamente la mitad de las posiciones posibles, y el diamante (V4) no es resoluble desde ninguna. Este resultado conecta directamente con el problema *complementario* clásico del *peg solitaire*: la viabilidad de una instancia depende fuertemente de las invariantes (paridad, coloreado módulo 3, función pagoda) inducidas por la posición del hueco inicial. La identificación empírica de qué huecos son resolubles aporta información estructural sobre cada variante que no se deduce trivialmente del tablero.

5) *Complementariedad con Fast Downward*: Una observación destacable es que nuestro sistema *complementa* a Fast Downward en lugar de competir con él. Fast Downward resuelve V3 y V4 (cruces simétricas) en pocos segundos, pero no consigue plan en V1, V2 ni V5 dentro de presupuestos razonables (timeouts de 60–300 s ensayados). En cambio, beam search con conectividad encuentra plan en V1 desde varios huecos iniciales (incluido el del enunciado), en V5 desde tres huecos del cuadrante representativo y, en general,

en las configuraciones del barrido para las que existe solución. La síntesis de ambos enfoques cubre toda la batería del enunciado: las cruces simétricas se delegan a Fast Downward (más rápido), y los tableros más densos (octógono, rombo) se resuelven con beam search más la heurística de conectividad.

6) *Por qué funciona la conectividad*: La conectividad funciona porque captura una invariante estructural del problema: para reducir un estado a una sola pieza, todas las piezas restantes deben permanecer en una *única* componente conexa hasta el final. Si una de las piezas queda aislada del resto (no hay manera de que sea saltada o salte), el plan ya es inviable. Minimizar el número de componentes conexas en cada paso fuerza a beam search a priorizar trayectorias donde el conjunto se mantiene cohesionado, lo que es una condición *casi necesaria* (aunque no suficiente) para alcanzar la meta. La pagoda, en cambio, ignora completamente la estructura topológica del estado: dos estados con el mismo número de piezas en posiciones del mismo peso reciben el mismo valor pagoda, aunque uno esté formado por una única cadena y el otro por varios grupos desconectados.

V. CONCLUSIONES

Se ha desarrollado un sistema completo de resolución del Senku basado en planificación automática. El dominio PDDL diseñado es genérico —soporta cualquier topología de tablero gracias a la codificación de la geometría como hechos del estado inicial— y se integra de forma transparente con la biblioteca `unified-planning` y con el planificador externo Fast Downward, siguiendo la metodología introducida en la Práctica 4. Sobre esa misma infraestructura se han implementado en Python una búsqueda en anchura (parte común) y el algoritmo *beam search* (parte específica de la convocatoria de junio), con varias heurísticas: la función pagoda, una variante compuesta y, sobre todo, una heurística de conectividad, además de reinicios estocásticos con desempate aleatorio.

Los experimentos arrojan tres resultados de interés. Primero, Fast Downward resuelve la cruz inglesa estándar (variante 1) en 31 movimientos en 33 s y demuestra de forma exhaustiva la irresolubilidad del cuadrado 5×5 con hueco central (variante 2). Segundo, *beam search* guiado por la función pagoda —pese a su admisibilidad— no resuelve ninguna variante, ni siquiera con $\beta = 2000$: el Senku presenta soluciones extremadamente raras (del orden de 10^6 planes en un espacio de 10^8 estados) y la pagoda no las discrimina. Tercero, y este es el hallazgo central, al sustituir la pagoda por una **heurística de conectividad** que minimiza el número de componentes conexas de piezas, *beam search* resuelve la cruz inglesa (31 movimientos, $\approx 7\,800$ nodos) y, eligiendo adecuadamente el hueco inicial, las tres variantes obligatorias (1, 3 y 5). El estudio de la posición del hueco confirma que las variantes 3 y 5 sólo admiten solución partiendo de un hueco en un brazo del tablero.

El estudio exhaustivo de huecos iniciales aporta, además, una caracterización empírica de la solubilidad de cada variante: la cruz inglesa admite plan desde cualquier hueco bajo simetría (12/12, 100%), el cuadrado 5×5 , el octogonal y la cruz extendida son parcialmente resolubles (44%, 46% y 44%

respectivamente), y el diamante es estructuralmente irresoluble. La variable `min_piezas_alcanzadas` confirma esta clasificación: en las variantes “parcialmente resolubles” beam search llega a 1 pieza en al menos un hueco, mientras que en el diamante el mínimo absoluto se estanca en 4 piezas en todos los huecos considerados.

A. Aplicabilidad metodológica

Más allá de los resultados específicos del Senku, las decisiones de diseño tomadas son aplicables a otros problemas de planificación con estructura similar:

- La codificación de la geometría como hechos estáticos del `:init` (en lugar de tipos o predicados de adyacencia) es útil en cualquier problema donde el tablero pueda variar entre instancias.
- La estrategia de *beam search iterativo* con β creciente reduce el sobre coste medio cuando algunas instancias son fáciles y otras difíciles, sin sacrificar cobertura.
- Heurísticas basadas en propiedades *estructurales* del estado (componentes conexas, propiedades topológicas) suelen superar a heurísticas *cuantitativas* (sumas ponderadas como la pagoda) en problemas con espacios de soluciones muy malos.

B. Líneas futuras

Como líneas futuras destacamos las siguientes:

- *Look-ahead* de uno o dos niveles para evaluar la heurística proyectada sobre los sucesores inmediatos en lugar de sobre el estado actual, lo que reduce el efecto *horizon* típico de las heurísticas locales.
- Combinación con búsquedas locales estocásticas (*hill climbing* con reinicios aleatorios), que pueden complementar beam search en regiones del espacio donde el haz se queda atascado.
- Integración explícita con planificadores como Fast Downward para resolver problemas más grandes y comparar la calidad de los planes encontrados por beam search (típicamente óptimos en número de movimientos, ya que el Senku tiene coste uniforme de 1 por jugada).
- Heurísticas específicas para el Senku basadas en *packings* y patrones combinatorios documentadas en la literatura clásica del juego [5], que podrían incorporarse al marco ProblemaSenku.
- Aplicación del marco a otros problemas de tipo solitario sobre rejillas (Mahjong solitario, juegos de eliminación), donde la conectividad también juega un papel central.
- Estudio de la simetría del tablero como invariante para reducir el espacio de búsqueda: la cruz inglesa tiene simetría de grupo D_4 (rotaciones y reflexiones), lo que permitiría factorizar el espacio en clases de equivalencia y reducir el factor de ramificación efectivo.

C. Reproducibilidad

Todo el código, los ficheros PDDL, los notebooks de experimentación y los CSV con los resultados están disponibles en el directorio `senku/` del entregable.

Los experimentos se reproducen ejecutando los scripts `senku/scripts/experimento_*.py`. La suite de tests unitarios (19 tests, todos en verde) verifica las propiedades formales (validez de la pagoda, monotonía, etc.) y la solubilidad efectiva de la cruz inglesa por beam search + conectividad.

D. Uso de inteligencia artificial generativa

Conforme exige el enunciado, declaramos el uso de un asistente generativo (Claude, modelo Opus 4.7) durante el desarrollo del trabajo. El asistente se ha empleado para:

- 1) Generar una estructura inicial del repositorio (módulos Python, organización de directorios) que ha sido revisada y adaptada por los autores.
- 2) Sugerir formulaciones alternativas del dominio PDDL y discutir las ventajas e inconvenientes de cada una, lo que ha influido en la decisión final de modelar la geometría como hechos `salto` del estado inicial.
- 3) Implementar las heurísticas, los algoritmos de búsqueda y los scripts de experimentación a partir de descripciones funcionales proporcionadas por los autores. El código resultante ha sido revisado, comprendido y, en algunos casos, modificado por los autores.
- 4) Asistir en la redacción de la memoria y la presentación, manteniendo siempre la responsabilidad editorial los autores.
- 5) Detectar y solucionar el bug de codificación de `up-fast-downward 0.5.2` mediante el módulo `parche_fd.py`.

Las consultas principales han sido del tipo: “implementa el algoritmo beam search siguiendo el pseudocódigo del enunciado”, “diseña una heurística que capture la conectividad de las piezas restantes en el tablero”, “escribe un script que pruebe sistemáticamente todas las posiciones del hueco inicial bajo simetría”, y “corrige el error de decodificación que aparece al llamar a Fast Downward en Windows”. En la defensa, los dos miembros del grupo seremos capaces de explicar y justificar todas y cada una de las decisiones técnicas y editoriales del entregable.

REFERENCIAS

- [1] G. Chaves Benítez, “Planificación automática aplicada al Senku. Propuesta de trabajo,” *Inteligencia Artificial — Ingeniería del Software*, Universidad de Sevilla, curso 2025/2026.
- [2] Planning Wiki, “Planning Domain Definition Language,” <https://planning.wiki/>, consultado en mayo de 2026.
- [3] A. Micheli *et al.*, “Unified-Planning: A Library Making Planning Technology Accessible,” en *Proc. of ICAPS*, 2022. Documentación: <https://unified-planning.readthedocs.io>.
- [4] M. Helmert, “The Fast Downward Planning System,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [5] J. C. Beasley, *The Ins and Outs of Peg Solitaire*, Oxford University Press, 1985.
- [6] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4ª ed., Pearson, 2020. Capítulos 3 y 4 sobre búsqueda en espacio de estados y búsqueda informada.
- [7] M. Ghallab *et al.*, “PDDL—The Planning Domain Definition Language,” Yale Center for Computational Vision and Control, Technical Report CVC TR-98-003, 1998.
- [8] M. Ghallab, D. Nau y P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.

- [9] D. S. Hochba (ed.), *Approximation algorithms for NP-hard problems*, PWS Publishing, 1997. Sobre la complejidad computacional de problemas combinatorios.
- [10] E. Berlekamp, J. H. Conway y R. Guy, *Winning Ways for your Mathematical Plays*, vol. 4, A. K. Peters, 2004. Capítulo sobre peg solitaire con la introducción original de la función pagoda por J. H. Conway.
- [11] N. G. de Bruijn, “A solitaire game and its relation to a finite field,” *Journal of Recreational Mathematics*, vol. 5, pp. 133–137, 1972. Análisis algebraico clásico del peg solitaire mediante argumentos de paridad y coloreado.